



DOCUMENTAR FLUXO DE BUILD E DEPLOY RESPONSIVO

Sistema de Visualização e Disseminação de Dados Limnológicos



1. Visão Geral do Projeto

O Sistema de Visualização e Disseminação de Dados Limnológicos é uma aplicação web desenvolvida com o objetivo de facilitar o acesso aberto a dados ambientais coletados por instituições de pesquisa. Esses dados são essenciais para estudos sobre o Balanço de Carbono em reservatórios brasileiros.

O projeto é uma parceria acadêmica entre instituições de ensino e pesquisa, integrando tecnologia, ciência ambiental e desenvolvimento web moderno.

Item Detalhes

- Instituição: FATEC Jacareí
- Curso: Desenvolvimento de Software Multiplataforma — 2º Semestre 2025
- Parceiros: INPE, UFRJ, UFJF, IIE
- Tema: Disseminação de dados limnológicos
- Metodologia: ABP (Aprendizagem Baseada em Projetos)
- Kick-off:
- Status: Em desenvolvimento

2. Objetivos do Sistema:

- Centralizar e disponibilizar dados limnológicos e meteorológicos em uma interface acessível.
- Democratizar o acesso a informações coletadas por campanhas e pelo sistema SIMA.
- Apoiar pesquisas científicas nacionais e internacionais relacionadas ao meio ambiente.
- Permitir visualizações dinâmicas e exportações para ferramentas analíticas externas.

3. Escopo e Funcionalidades:

3.1 Requisitos Funcionais:

Código	Descrição
RF01	Visualizar parâmetros armazenados, filtrando por instituição, reservatório e período
RF02	Consultar dados em formato tabular
RF03	Exportar dados no formato CSV
RF04	Exibir localização dos dados em mapa interativo
RF05	Visualizar séries temporais em gráficos interativos

3.2 Requisitos Não Funcionais:

Código	Descrição
RNF01	Interface intuitiva e acessível para usuários não técnicos
RNF02	Desempenho otimizado com carregamento ágil
RNF03	Interface seguindo padrões visuais do INPE

3.3 Restrições de Projeto

Código	Descrição
RP01	Banco de dados em PostgreSQL.
RP02	Back-end em Node.js com TypeScript.
RP03	Front-end em React com TypeScript.
RP04	Aplicação containerizada (Docker).

4. Arquitetura Técnica:

4.1 Tecnologias:

- Frontend: React + Vite + TypeScript + styled-components
- Backend: Node.js + Express + TypeScript
- Banco de Dados: PostgreSQL (um container por domínio: Furnas, BALCAR, SIMA)
- Infraestrutura: Docker + Docker Compose
- Metodologia de Desenvolvimento: Scrum/Agile

4.2 Organização do Projeto:

```
app/  
  
├── balcar-campanha/  # Dados BALCAR  
  
├── furnas-campanha/  # Dados Furnas  
  
├── sima/             # Dados SIMA  
  
├── server/           # API em Node.js  
  
├── front/            # Interface React  
  
├── .github/workflows/ # (Planejado) CI/CD  
  
└── docker-compose.dev.yml
```

5. Pipeline de Desenvolvimento:

5.1 Fluxo CI/CD (Planejado):

- 1. Push ou Pull Request na branch main
- 2. Rodar lint, build e testes
- 3. Implantar em ambiente de homologação
- 4. Aprovação manual → deploy em produção

5.2 Etapas do Build:

Front-end: build Vite + minificação e otimização de assets

Back-end: build TypeScript + transpile + hot reload

Banco de Dados: scripts SQL CREATE TABLE e COPY executados automaticamente no start

Containers: orquestrados via docker-compose.dev.yml

6. Validação de Tasks:

Todas as tasks do projeto foram validadas conforme os requisitos estabelecidos no Product Backlog e os critérios definidos pelo time e professores.

Passos de Validação:

- 1. Verificar se a task foi commitada no repositório do grupo.
- 2. Ir até a estação de trabalho do integrante para entender a execução técnica.
- 3. Analisar dúvidas e soluções implementadas.
- 4. Testar o comportamento no ambiente local e via Docker.
- 5. Validar conforme Definition of Done:
 - Código lintado e padronizado
 - Build sem erros
 - Endpoint ou componente documentado
 - Teste manual do fluxo principal concluído
 - Logs limpos de erros

7. Referência de API

Rotas Base:

Recurso	Endpoint base	Descrição
SIMA	\sima	Dados automáticos de séries temporais
FRUNAS	\furnas	Dados coletados em campanhas
BALCAR	\balcar	Dados coletados em campanhas BALCAR

Exemplo:

GET /sima/sima/all?page=1&limit=20

Respostas paginadas respeitam PAGE_SIZE padrão.

CORS configurado para permitir acesso do front-end.

8. Exportação de Dados CSV:

A aplicação conta com parser CSV próprio, desenvolvido em TypeScript.

Recursos:

- Extração dinâmica do banco PostgreSQL
- Cabeçalhos automáticos e metadados
- Filtros aplicados pelo usuário
- Exportação compatível com Excel, LibreOffice, R, Python e outras ferramentas analíticas.
- Codificação UTF-8 com BOM:
 - >  Arquivos exportados incluem metadados (início do arquivo, com #).

9. Critérios de Qualidade:

- Requisitos RF01–RF05 implementados.
- Tempo de resposta otimizado nas consultas.
- Mensagens de erro amigáveis.
- Compatibilidade com navegadores modernos.
- Logs sem erros críticos.
- Responsividade e acessibilidade básicas implementadas.

10. Artefatos Scrum

Artefato	Conteúdo
Product Backlog	Requisitos e funcionalidades planejadas
Sprint Backlog	Tarefas de cada Sprint
Burndown Chart	Progresso da Sprint
Dailys	ATAs de reuniões diárias

Todos os artefatos estão organizados na pasta /Scrum/ no repositório do projeto.

11. Troubleshooting — Problemas Comuns

Problema	Causa provável	Solução
Porta ocupada	Processo ativo em 5173/3000	Encerrar processo ou alterar porta
API não responde	Porta incorreta	Verificar .env
Erro de CORS	Origem não autorizada	Ajustar CORS_ORIGIN
CSV vazio	Filtros aplicados sem dados	Remover ou ajustar filtros

12. Considerações Finais:

O projeto cumpre os objetivos propostos de disseminação de dados limnológicos, apresentando uma arquitetura modular, escalável e documentada, que pode evoluir para integrações com sistemas externos de análise ambiental.

Próximos passos:

- Implantação de CI/CD automatizado
- Melhoria de acessibilidade e responsividade
- Implementação de testes automatizados
- Publicação em ambiente de produção.

Versão da Documentação: 1.0

Última Atualização: 22/10/2025

Responsável Técnico: Equipe ABP 2DSM — FATEC Jacareí